

rsedit — Rapporto sui problemi del codice

Esito dell'analisi della revisione `08a853d`, tenuto separato dai manuali di architettura

Indice

1	Che cosa contiene questo rapporto	2
1.1	Come leggere una scheda	2
1.2	Come è stato condotto l'esame	2
1.3	Riepilogo	3
2	Lettore	6
3	Valutatore e libreria di base dell'interprete	8
4	Buffer, modifiche e annullamento	14
5	Tastiera, comandi e primitive dell'editor	15
6	Minibuffer	17
7	File e salvataggio	18
8	Avvio, configurazione e registro	20
9	Lavoro in background	22
10	Modi, sintassi e colorazione	23
11	Ricerca, sostituzione e liste di risultati	25
12	Comandi di shell	27
13	Disegno e terminale	28
14	Documentazione, commenti, codice morto, build e test	30
15	Appendici	32
15.1	Differenze da Emacs che non sono difetti	32
15.2	Parti esaminate e trovate solide	32

1 Che cosa contiene questo rapporto

Questo documento elenca i problemi trovati analizzando il codice di `rsedit`. È separato di proposito dai due manuali di architettura, `architettura-interprete.typ` e `architettura-editor.typ`. I manuali descrivono come il codice funziona e restano validi anche dopo una correzione; questo rapporto descrive dove il codice non fa ciò che dovrebbe, e ogni sua scheda è destinata a sparire quando il problema viene risolto.

Ogni problema ha un identificativo stabile, formato dalla sigla dell'area e da un numero (`LET-1`, `BUF-2` ...), che si può citare in un commit o in una discussione. Le schede sono generate da un'unica tabella di dati all'inizio del sorgente: per aggiungerne una, o per toglierne una risolta, basta modificare quella tabella, e il riepilogo si aggiorna da solo.

1.1 Come leggere una scheda

Ogni scheda riporta:

- **Gravità**, secondo la scala qui sotto.
- **Dove**: file e righe della revisione analizzata. I percorsi senza prefisso sono relativi a `core/src/`; quelli che cominciano con `src/` riguardano la TUI e quelli con `core/lisp/` i moduli Lisp.
- **Verifica**: «verificato» se il comportamento è stato riprodotto eseguendo il codice (il testo dice come), «dal codice» se la conclusione viene dalla sola lettura.
- **Cosa succede**, il meccanismo del difetto; **Come riprodurlo**, quando serve; **Effetto**, ciò che l'utente vede; **Correzione proposta**.

Gravità	Significato
ALTA	L'editor termina, si blocca senza via d'uscita o perde dati; oppure un contenuto esterno può far eseguire azioni al terminale; oppure l'editor finisce in uno stato da cui si esce solo riavviando.
MEDIA	Un comportamento sbagliato o un rallentamento di secondi in un uso realistico, senza perdita di dati.
BASSA	Casi limite, incoerenze, codice morto, commenti e documentazione superati.

1.2 Come è stato condotto l'esame

Il codice è stato letto per intero: l'interprete (`core/src/lisp`), la libreria dell'editor (`core/src`), la TUI (`src/`) e i moduli Lisp (`core/lisp`). Ogni comportamento sospetto è stato riprodotto, dove possibile, con piccoli programmi esterni che usano `rsedit_core` come una libreria:

- un **esecutore di scenari** che valuta espressioni Lisp, preme tasti attraverso il percorso reale dei tasti, attende i lavori di background e stampa buffer, area messaggi ed evidenziazioni della fotografia dello schermo;
- un **fuzzer delle primitive**, che chiama ciascuna delle 628 funzioni registrate (tranne quelle che toccano file o processi esterni) con 807 combinazioni di argomenti anomali (zero, negativi, 10^{23} , infinito, NaN, stringhe vuote e non ASCII, liste puntate, vettori, mappe, lambda), una volta su un buffer vuoto e una su un buffer con testo multiriga, caratteri non ASCII, tab, una regione attiva e finestre divise;
- un **fuzzer del lettore**, su tutte le stringhe fino a tre caratteri di un alfabeto di 28 simboli sintattici e su 300 000 stringhe casuali più lunghe;
- un **fuzzer delle forme speciali**, che ha valutato 181 322 forme speciali con sintassi malformata (liste di legami sbagliate, parametri impossibili, `yield` fuori posto) senza trovare alcun panic.

Le misure di tempo sono state prese con build di release su una macchina condivisa, quindi vanno lette come ordini di grandezza.

1.3 Riepilogo

In totale **80 problemi**: **11 di gravità alta**, 31 di gravità media e 38 di gravità bassa; 49 sono stati riprodotti eseguendo il codice.

I problemi di gravità alta hanno quasi tutti la stessa forma: un errore in un punto che dovrebbe restituire un errore Lisp (il lettore, un argomento numerico, un colore) diventa un panic o un ciclo infinito. Poiché né il thread dei comandi, né quello di background, né la TUI intercettano i panic, l'effetto è sempre il più grave possibile. Tre correzioni trasversali ridurrebbero da sole l'impatto di gran parte dell'elenco:

1. una guardia che ripristini il terminale su panic ed errori ([TUI-2](#));
2. un `catch_unwind` attorno a ogni turno dello scheduler ([BKG-1](#)) e, nel thread dei comandi, attorno a `run_command_form`, trasformando un panic in un errore del comando;
3. un limite di profondità e di allocazione nelle operazioni guidate dall'utente ([INT-2](#), [CMD-3](#), [INT-13](#)).

La tabella seguente elenca tutti i problemi, dal più grave; l'identificativo rimanda alla scheda.

ID	Problema	Gravità	Verifica
LET-1	Input che finisce dentro un simbolo con una lista aperta: ciclo infinito, memoria esaurita	ALTA	verificato
LET-2	<code>.</code> o una parentesi chiusa dopo un apice: panic del lettore	ALTA	verificato
INT-1	<code>unwind-protect</code> ricarica il carburante a ogni esecuzione: ciclo infinito inarrestabile	ALTA	verificato
INT-2	Ricorsione profonda: stack overflow e chiusura del processo	ALTA	verificato
BUF-1	<code>undo / redo</code> scavalcano le porte: sola lettura, versione, cache, overlay	ALTA	verificato
MIN-1	Prompt annidati: testo perso e finestra flottante fantasma	ALTA	verificato
FIL-1	<code>save-buffer</code> fallito: solo una riga nel registro	ALTA	verificato
AVV-1	<code>rsedit.log</code> nella directory corrente: senza permessi l'editor non parte	ALTA	verificato
BKG-1	Lo scheduler muore al primo panic: niente colore, autosave, worker	ALTA	verificato
TUI-1	Caratteri di controllo stampati così come sono: iniezione di sequenze nel terminale	ALTA	verificato
TUI-2	Nessun ripristino del terminale su panic o errore	ALTA	dal codice
LET-3	Annidamento profondo: stack overflow nel lettore	MEDIA	verificato
INT-3	<code>setq</code> su una variabile non definita dentro una funzione o un <code>let</code> crea una locale	MEDIA	verificato
INT-4	Annidamento profondo in valutazione, rilascio, <code>equal</code> e stampa: stack overflow	MEDIA	verificato
INT-5	<code>dotimes</code> con corpo vuoto: nessun carburante consumato	MEDIA	verificato
INT-6	Cicli di <code>Arc</code> tra chiusure e ambienti: memoria persa	MEDIA	verificato
INT-7	<code>(eq x x)</code> falso per <code>lambda</code> , atomi, fiber e primitive	MEDIA	verificato
INT-8	Una fiber non è <code>eq</code> né <code>equal</code> a sé stessa	MEDIA	verificato
INT-9	<code>resume</code> di una fiber in esecuzione la marca come finita	MEDIA	verificato
INT-10	<code>yield</code> dentro <code>condition-case</code> : catturato come errore, la fiber non si sospende	MEDIA	verificato
INT-11	Backquote: coda puntata, mappe e annidamento non elaborati	MEDIA	verificato
INT-12	<code>format</code> interpreta <code>\b</code> , <code>\U</code> , <code>\.</code> ... a tempo di esecuzione	MEDIA	verificato
INT-13	<code>make-string</code> senza limite: panic o abort	MEDIA	verificato
BUF-2	<code>GapBuffer::cursor_pos</code> lineare nella posizione	MEDIA	verificato
CMD-1	Tasti non ASCII: non digitabili, non legabili	MEDIA	verificato
CMD-2	<code>KeyCode</code> senza Delete/Home/End/PgUp/PgDn/F1...	MEDIA	dal codice

ID	Problema	Gravità	Verifica
CMD-3	Conteggi e larghezze senza limite nelle primitive Rust: panic, abort, blocchi	MEDIA	verificato
MIN-2	C-g nel minibuffer non chiude il prompt	MEDIA	verificato
FIL-2	Scrittura non atomica dei file	MEDIA	dal codice
FIL-3	Aprire un file non UTF-8 fallisce in silenzio	MEDIA	verificato
AVV-2	eval_file: errori di sintassi silenziosi; un commento finale senza a capo annulla il file	MEDIA	verificato
AVV-3	Scrittura del registro con .expect: panic con disco pieno	MEDIA	dal codice
SIN-1	Regioni che corrispondono a «»: ciclo infinito o memoria esaurita nell'evidenziatore	MEDIA	verificato
SIN-2	Colorazione quadratica sulle righe lunghe	MEDIA	verificato
SIN-6	C-M-f a metà di un simbolo: un'espressione di troppo	MEDIA	verificato
RIC-1	replace-regexp con match vuoti: tutto nello stesso punto	MEDIA	verificato
RIC-2	C-M-r: tempo quadratico per tasto	MEDIA	verificato
SHL-1	stderr letto dopo stdout: blocco con più di 64 KB su stderr	MEDIA	verificato
SHL-2	Output non UTF-8: lettura interrotta, comando ucciso da SIGPIPE	MEDIA	verificato
TUI-3	Tab non espansi: cursore, colori e riempimento sfasati	MEDIA	verificato
TUI-4	CJK ed emoji contati come una colonna	MEDIA	dal codice
TUI-5	Color::parse: panic su colori non ASCII	MEDIA	verificato
LET-4	.9, 1e5, +5 letti come simboli	BASSA	verificato
LET-5	\r, " e , dentro i simboli; #'f e ?a simboli	BASSA	dal codice
LET-6	todo!() è stato sporco dopo NumberParseError	BASSA	dal codice
INT-14	Interi oltre 2 ⁶³ stampati come 9223372036854775807	BASSA	verificato
INT-15	add-to-list: ambiente del chiamante, terzo argomento, O(n·m)	BASSA	verificato
INT-16	memq/assq usano equal; member copia la coda	BASSA	dal codice
INT-17	string-to-number accetta inf/NaN; substring oltre la fine dà «»	BASSA	dal codice
INT-18	spawn restituisce una Form vuota	BASSA	verificato
INT-19	bootstrap_vm lascia verify-math, verify-fiber, fiber-step	BASSA	verificato
INT-20	Errore UncorrectFunctionDefinition fuorviante	BASSA	verificato
INT-21	Atomi senza compare-and-swap	BASSA	dal codice
INT-22	Parametri t/nil irraggiungibili	BASSA	dal codice
INT-23	unreachable! in function-doc	BASSA	dal codice
BUF-3	GapBuffer::clone è todo!(); find_forward underflow	BASSA	dal codice
CMD-4	parse_key: un solo prefisso; define-key fallisce in silenzio	BASSA	verificato
CMD-5	C-u 3 x inserisce una sola x	BASSA	verificato
CMD-6	Argomento n non numerico → 0; conteggi negativi → 0	BASSA	dal codice
MIN-3	set_minibuffer_content scavalca le porte	BASSA	dal codice
FIL-4	Corsa tra copia del testo e azzeramento di «modificato»	BASSA	dal codice
FIL-5	Autosave: copia preventiva; versione segnata anche se la scrittura fallisce	BASSA	dal codice
AVV-4	Log::lines senza limite	BASSA	dal codice
AVV-5	init.lisp predefinito scritto solo alla prima esecuzione	BASSA	dal codice

ID	Problema	Gravità	Verifica
SIN-3	<code>write-file</code> con cambio di modo: colori vecchi	BASSA	dal codice
SIN-4	<code>make-mode</code> ricrea il modo, perdendo legami e hook	BASSA	dal codice
SIN-5	Facce: registro globale senza limite, indice <code>u16</code>	BASSA	dal codice
RIC-3	Una sostituzione = un passo di <code>undo</code>	BASSA	verificato
RIC-4	Match oltre il limite scartato; colonna ricalcolata per ogni match	BASSA	dal codice
RIC-5	Risultati per numero di riga, non per marcatore	BASSA	dal codice
RIC-6	<code>isearch-exit</code> : mark sull'altra estremità del match	BASSA	verificato
SHL-3	Nessuna primitiva per interrompere un comando di shell	BASSA	dal codice
SHL-4	<code>manpage</code> : nome interpolato senza quoting nella shell	BASSA	dal codice
TUI-6	Finestre flottanti senza sintassi e overlay	BASSA	dal codice
TUI-7	<code>%p</code> = <code>All</code> solo con una riga	BASSA	dal codice
TUI-8	Niente schermo alternativo: all'uscita lo schermo viene cancellato	BASSA	dal codice
DOC-1	Commenti e docstring superati	BASSA	dal codice
DOC-2	<code>local_keymap</code> mai usato; Debug di <code>EditorState</code> è <code>todo!()</code>	BASSA	dal codice
DOC-3	<code>build.rs</code> : percorso di <code>target</code> presunto, panic se manca	BASSA	verificato
DOC-4	<code>perf::suite</code> instabile; <code>cargo clippy --all-targets</code> fallisce	BASSA	verificato

Area	Descrizione	Alta	Media	Bassa
LET	Lettore	2	1	3
INT	Valutatore e libreria di base dell'interprete	2	11	10
BUF	Buffer, modifiche e annullamento	1	1	1
CMD	Tastiera, comandi e primitive dell'editor	0	3	3
MIN	Minibuffer	1	1	1
FIL	File e salvataggio	1	2	2
AVV	Avvio, configurazione e registro	1	2	2
BKG	Lavoro in background	1	0	0
SIN	Modi, sintassi e colorazione	0	3	3
RIC	Ricerca, sostituzione e liste di risultati	0	2	4
SHL	Comandi di shell	0	2	2
TUI	Disegno e terminale	2	3	3
DOC	Documentazione, commenti, codice morto, build e test	0	0	4

Tabella 1: Distribuzione per area e gravità

2 Lettore

LET-1 — Un input che finisce dentro un simbolo, con una lista aperta, esaurisce la memoria

Gravità ALTA

Dove `lisp/parser.rs:456` (gestione della fine dell'input negli stati `InSymbol` e `InNumberMinusStart`); il ciclo che non termina è `parse_list` a `lisp/parser.rs:492`

Verifica Riprodotto con `eval-string-safe` e con `M-:` nell'editor; trovato dal fuzzer del lettore.

Cosa succede. Quando l'input finisce mentre il lessico sta leggendo un simbolo, `next_token` restituisce il simbolo accumulato ma **non** riporta lo stato a `Default`. Alla chiamata successiva l'input è ancora finito e lo stato è ancora `InSymbol`, quindi restituisce un altro simbolo, vuoto; e così ad ogni chiamata. `parse_list` aspetta una `)` o la fine dei token (`Token::Void`), riceve invece un flusso infinito di simboli vuoti e li accoda tutti. Per un vettore il ciclo è identico; per una mappa (`{a}`) il ciclo gira senza fine anche senza crescita evidente della memoria.

Come riprodurlo.

```
(eval-string-safe "(foo") ; abort: memoria esaurita
(eval-string-safe "(a (b c)") ; idem, anche "[x", "(-", "'(x"
(eval-string-safe "(foo ") ; corretto: errore UnclosedList
;; nell'editor: M-: (message RET
```

Effetto. La memoria cresce di centinaia di megabyte al secondo finché il processo termina («memory allocation of 1610612736 bytes failed») o interviene l'OOM killer del sistema; il terminale resta in raw mode ([TUI-2](#)) e il lavoro non salvato è perso. Basta dimenticare una parentesi in `M-:`. Sono esposti `eval-string`, `eval-string-safe` (quindi `M-:`) e il caricamento dei temi, che passa il testo del file a `eval-string` (`lisp/theme.lisp:101`). `eval_file` non è esposto: avvolge il contenuto in `(progn ...)`, e la parentesi aggiunta chiude il simbolo.

Correzione proposta. Riportare `lexer_state` a `Default` in ogni ramo del codice dopo il ciclo che restituisce un token. Come difesa in profondità, `parse_list`, `parse_vector` e `parse_map` possono rifiutare un simbolo vuoto. Il test che avrebbe trovato il difetto: per ogni prefisso di un'espressione valida il lettore deve terminare, con un valore o con un errore.

LET-2 — Un token che non può cominciare un'espressione manda il lettore in panic

Gravità ALTA

Dove `lisp/parser.rs:642` (`_ => unreachable!(...) in Parser::next`)

Verifica Riprodotto (uscita 101). Il fuzzer del lettore, su 322 764 stringhe, l'ha trovato 7 477 volte con `Dot`, 295 con `RSquared`, 288 con `RParen` e 8 con `RBracket`.

Cosa succede. `Parser::next` costruisce un'espressione a partire da simboli, stringhe, numeri, parentesi aperte e prefissi (`'`, ```, `,`, `,`, `@`). Ogni altro token in posizione di espressione finisce nel ramo `_ => unreachable!`, che è un panic e non un errore. Succede con un punto fuori dalla coda di una lista e con una parentesi chiusa subito dopo un prefisso.

Come riprodurlo.

```
(eval-string-safe "[1 . 2]") ; panic: token parse not implemented for Dot
(eval-string-safe "(list 'a ')") ; panic: ... for RParen
;; anche: "{a .}" ". a" "'.'" "[1 `]" "(a ,)" "{:k ,@}"
```

Effetto. Il panic non è un errore Lisp, quindi `eval-string-safe` e `condition-case` non lo intercettano. Nell'editor `M-: (list 'a ')` termina il programma; in un lavoro di background uccide il thread condiviso ([BKG-1](#)).

Correzione proposta. Una variante `ParserError::UnexpectedToken` restituita al posto del panic, più i casi elencati sopra nei test del lettore.

LET-3 — Un annidamento profondo esaurisce lo stack del lettore

Gravità **MEDIA**

Dove `lisp/parser.rs` — `next` → `parse_list` → `next`, un livello di ricorsione Rust per ogni parentesi

Verifica In release `'((((...)))` con 20 000 livelli viene letto, con 50 000 il processo abortisce.

Cosa succede. Il lettore discende ricorsivamente: ogni parentesi aperta è una chiamata Rust in più, senza un contatore di profondità.

Effetto. Uno stack overflow non è intercettabile: il processo termina. Un file di dati generato, o una stringa passata a `eval-string`, basta a chiudere l'editor.

Correzione proposta. Un contatore di profondità nel lettore con un errore oltre un limite (per esempio 10 000), oppure un lettore iterativo con una pila esplicita.

LET-4 — `.9`, `1e5` e `+5` sono letti come simboli

Gravità **BASSA**

Dove `lisp/parser.rs:388` (`'0'..'9'`, intervallo che esclude il 9), stati `InNumber*`

Verifica Riprodotto; `clippy` segnala `almost_complete_range` proprio alla riga 388.

Cosa succede. Dopo un punto iniziale il lessico accetta solo le cifre da 0 a 8, quindi `.5` è un numero e `.9` un simbolo. La notazione esponenziale non esiste: `1e5`, `2.5e-3` e `1e300` sono simboli, e valutarli dà `UnboundVariable`. Anche `+5` è un simbolo. `string-to-number`, che usa `f64::from_str`, accetta invece `"1e5"`, `"inf"` e `"NaN"` ([INT-17](#)).

Correzione proposta. `'0'..'9'`; esponente e segno `+` nello stato dei numeri; stesse regole per il lettore e per `string-to-number`.

LET-5 — Stranezze del lessico dei simboli

Gravità **BASSA**

Dove `lisp/parser.rs:206` (`'\r' => {}` nello stato `InSymbol`), `lisp/parser.rs:190-210`

Verifica Dalla lettura del codice.

Cosa succede. Dentro un simbolo un `\r` viene saltato senza chiuderlo (`foo\rbar` diventa `foobar`); `"` e `,` ne fanno parte (`foo"bar"` è un unico simbolo, così come `a,b`). `#'f` e `?a` sono simboli e non la funzione `f` o il carattere `a` come in Emacs Lisp.

Correzione proposta. Trattare `\r` come spazio, chiudere il simbolo su `"`, `'`, ``` e `,`; decidere esplicitamente se `#'` e `?` vanno supportati.

LET-6 — Un `todo!()` nel lettore e uno stato sporco dopo un numero non valido

Gravità **BASSA**

Dove `lisp/parser.rs:446` (`todo!`), `lisp/parser.rs:381-384`

Verifica Dalla lettura del codice.

Cosa succede. Il ramo `Default` del codice dopo il ciclo chiama `todo!()` se il token non è vuoto e non è una parentesi chiusa; oggi sembra irraggiungibile. Dopo un `NumberParseError` nello stato `InNumberAfterDot` lo stato e il token non vengono ripristinati, quindi una nuova chiamata a `next()` sullo stesso `Parser` riparte da uno stato sporco.

Correzione proposta. Sostituire il `todo!` con un errore; azzerare token e stato prima di restituire ogni errore.

3 Valutatore e libreria di base dell'interprete

INT-1 — Un ciclo che contiene `unwind-protect` non esaurisce mai il carburante

Gravità ALTA

Dove `lisp/eval.rs:991` (`ctx.begin_unwind()` chiamato sempre), `editor/mod.rs:287-291` (`grant(100_000)`), `lisp/fuel.rs:139` (`grant` porta il residuo ad **almeno** la quantità data)

Verifica Riprodotto: `(let ((x 0)) (while (< x 2000000) (unwind-protect (setq x (+ x 1)))) x)` restituisce 2 000 000, mentre lo stesso ciclo senza `unwind-protect` si ferma per carburante.

Cosa succede. Il carburante è l'unica difesa contro un ciclo infinito durante un comando: mentre un comando gira l'editor non legge la tastiera, quindi `C-g` non può interromperlo. `unwind-protect` chiama `begin_unwind` dopo il corpo **sempre**, anche quando il corpo è finito normalmente, e `begin_unwind` alza il carburante rimasto ad almeno 100 000 unità. Un ciclo con un `unwind-protect` nel corpo riceve quindi una ricarica a ogni giro e non scende mai sotto quella soglia.

Come riprodurlo.

```
(while t (unwind-protect nil)) ; non termina mai
```

Effetto. L'editor resta bloccato per sempre al primo ciclo infinito che attraversa un `unwind-protect`, anche se questo sta in una funzione di libreria chiamata dal ciclo. L'unica uscita è uccidere il processo, perdendo ciò che non è stato salvato.

Correzione proposta. Chiamare `begin_unwind` solo quando il corpo è fallito con `OutOfFuel`. In alternativa, concedere la ricarica al più una volta per comando, con un indicatore nel `FuelMeter` azzerato da `begin`.

INT-2 — Nessun limite alla profondità della ricorsione: lo stack si esaurisce e il processo termina

Gravità ALTA

Dove `lisp/eval.rs` — ogni chiamata che non è in coda è una ricorsione Rust (`eval` → `eval_special_form_or_call_step` → `eval` degli argomenti)

Verifica Riprodotto: in debug il processo termina tra 200 e 400 livelli, in release tra 5 000 e 6 000 sul thread principale (8 MB di stack). Sui thread dei worker e di `spawn` (2 MB) il limite è circa quattro volte più basso.

Cosa succede. Il trampolino elimina la crescita dello stack solo per le chiamate in coda. Una chiamata in posizione di argomento discende nello stack Rust, e non esiste un contatore di profondità.

Come riprodurlo.

```
(defun f (n) (if (= n 0) 0 (+ 1 (f (- n 1)))))  
(f 10000) ; stack overflow: il processo termina
```

Effetto. Un errore banale in una funzione ricorsiva scritta dall'utente chiude l'editor senza un messaggio e senza salvare. Uno stack overflow non è un errore Lisp, quindi nessun `condition-case` lo intercetta. Emacs, con `max-lisp-eval-depth`, solleva invece un errore.

Correzione proposta. Un contatore di profondità per thread, come quello del carburante, incrementato all'ingresso di una chiamata di funzione, con un errore oltre un limite tarato sul thread con meno stack. In aggiunta, i thread dei worker e di `spawn` possono essere creati con `std::thread::Builder::stack_size` per avere lo stesso margine del thread principale.

INT-3 — `setq` di una variabile mai definita crea una variabile locale che sparisce

Gravità MEDIA

Dove `lisp/eval.rs` (ramo `setq: update_variable`, altrimenti `set_variable` nell'ambiente corrente), `lisp/environment.rs`

Verifica Riprodotto: dopo `(defun g () (setq zz 5))` e `(g)`, `zz` è ancora non definita. Lo stesso accade dentro un `let` e nel corpo di una `fiber`. Al livello più esterno `setq` funziona.

Cosa succede. `setq` cerca la variabile lungo la catena degli ambienti. Se non la trova, la crea nell'ambiente **corrente**, cioè in quello della funzione, del `let` o della `fiber` che sta eseguendo, e non nell'ambiente globale.

Quando la funzione ritorna, la variabile sparisce. In Emacs Lisp un `setq` su una variabile non legata imposta il valore globale.

Come riprodurlo.

```
(defun conta () (setq contatore 1))  
(conta)  
contatore                ; => UnboundVariable
```

Effetto. Una configurazione o un modulo che fanno `setq` di uno stato globale dentro una funzione, senza un `defvar` precedente, perdono quello stato in silenzio. È anche la causa per cui `add-to-list` chiamato dentro una funzione non modifica la lista globale (INT-15).

Correzione proposta. Se la variabile non esiste in nessun ambiente, crearla nell'ambiente radice, come fa Emacs. Se invece la semantica attuale è voluta, va documentata in `setq` e va reso disponibile un modo esplicito (`set-default` o `defvar`) da usare dentro le funzioni.

INT-4 – Annidamenti profondi esauriscono lo stack anche fuori dal lettore

Gravità MEDIA

Dove `lisp/eval.rs` (forme annidate); `lisp/types.rs` (`Drop` di `ConsCell`, iterativo solo lungo il `cdr`); `lisp/lispexp.rs` (`PartialEq` ricorsivo sul `car`, `Debug` ricorsivo)

Verifica In release: `(list (list ...))` annidate abortiscono tra 5 000 e 10 000 livelli; liste annidate nel `car` abortiscono a 300 000 livelli nel rilascio della memoria e in `equal`.

Cosa succede. Il rilascio di una lista e il confronto con `equal` scorrono la catena dei `cdr` in modo iterativo, ma scendono nei `car` per ricorsione. La valutazione di forme annidate e la stampa sono ricorsive.

Effetto. Abort del processo. È meno probabile del caso del lettore (LET-3), perché servono strutture costruite apposta, ma una struttura dati profonda creata in un ciclo basta.

Correzione proposta. Rilascio, `equal` e stampa con una pila esplicita; per la valutazione vale la correzione di INT-2.

INT-5 – `dotimes` con il corpo vuoto non consuma carburante

Gravità MEDIA

Dove `lisp/eval.rs:866-910`: il ciclo `while i < count` chiama `eval` solo per le forme del corpo

Verifica Riprodotto: `(dotimes (i 300000000))` gira 300 milioni di volte (24 s in release) invece di fermarsi; con un corpo qualsiasi, anche `nil`, il ciclo si ferma con `OutOfFuel`.

Cosa succede. Il carburante si consuma in `eval_step`. Il ciclo di `dotimes` è scritto in Rust e, senza forme nel corpo, non chiama mai `eval`.

Effetto. Con un conteggio grande l'editor resta bloccato per ore. Il caso è raro, ma è una via d'uscita dal meccanismo che dovrebbe garantire che ogni comando termini.

Correzione proposta. Consumare un'unità di carburante a ogni giro del ciclo, come fa `while`. Lo stesso vale per `dolist`.

INT-6 – Le chiusure che si riferiscono a sé stesse non vengono mai liberate

Gravità MEDIA

Dove `lisp/types.rs` (una `Lambda` possiede il suo ambiente con un `Arc`), `lisp/environment.rs`

Verifica Riprodotto: `(dotimes (i N) (let ((f nil)) (setq f (lambda () 1)) nil))` porta il picco di memoria a 200 MB con `N = 300 000` e a 400 MB con `N = 600 000`, circa 650 byte per giro. Con `(funcall (lambda () 1))` resta a 10 MB.

Cosa succede. Una `lambda` tiene in vita l'ambiente in cui è nata. Se quell'ambiente contiene a sua volta la `lambda`, si forma un ciclo di contatori di riferimenti che non scende mai a zero. Succede quando una `lambda` viene assegnata a una variabile del `let` che la crea, quando una callback è salvata nel `let` che la definisce, o con un `defun` dentro una funzione. Non esiste un garbage collector che rompa questi cicli.

Effetto. Ogni ripetizione di un idiomma comune perde memoria. In una sessione lunga, con hook e callback che creano chiusure, la memoria cresce senza limite.

Correzione proposta. Un raccoglitore di cicli, per esempio un mark-and-sweep periodico degli ambienti raggiungibili dall'ambiente globale e dalle pile delle fiber. Nel frattempo, documentare l'idioma da evitare.

INT-7 — `eq` è falso per una `lambda`, un `atomo`, una `fiber` o una `primitiva` confrontati con sé stessi

Gravità **MEDIA**

Dove `lisp/base/predicates.rs:13-33` (`primitive_eq_impl` ha rami solo per numeri, simboli, `nil`, forme, cons, vettori, mappe e stringhe)

Verifica Riprodotto: `(let ((f (lambda () 1))) (eq f f))` dà `nil`; lo stesso per un atomo. `equal` dà `t`, tranne che per le fiber (INT-8).

Cosa succede. Per i tipi senza un ramo dedicato `eq` cade nel caso generico e restituisce `nil`, anche quando i due argomenti sono lo stesso oggetto.

Effetto. Viene violata la legge più elementare dell'identità. Il codice che cerca una funzione o un atomo per identità non la trova.

Correzione proposta. Un ramo `Arc::ptr_eq` per `Lambda`, `Atom`, `Fiber` e `Primitive`.

INT-8 — Una `fiber` non è uguale nemmeno a sé stessa

Gravità **MEDIA**

Dove `lisp/types.rs:179-183`: `std::ptr::eq(self, other)` confronta gli indirizzi dei due involucri, non l'`Arc` condiviso

Verifica Riprodotto: `(setq f (fiber (yield 1)))` e poi `(list (eq f f) (equal f f) (memq f (list f)))` dà `(nil nil nil)`.

Cosa succede. Ogni volta che una fiber viene letta da una variabile se ne clona l'involucro, cioè l'`Arc`. Due cloni hanno indirizzi diversi, quindi il confronto per indirizzo dell'involucro è sempre falso.

Effetto. Una fiber non si trova in nessuna lista: `memq`, `member`, `assoc` e `delete` falliscono, quindi un registro di fiber non può toglierne una.

Correzione proposta. `Arc::ptr_eq(&self.0, &other.0)`.

INT-9 — `resume` di una `fiber` che sta già girando la uccide

Gravità **MEDIA**

Dove `lisp/base/fibers.rs:52-90`

Verifica Riprodotto: con `(setq f (fiber (resume f) (yield 1) 2))`, `(resume f)` dà 1 e `(fiber-done-p f)` dà `t`; un nuovo `(resume f)` dà `nil` invece di 2.

Cosa succede. Mentre una fiber gira, `resume` le toglie la pila dei frame, così un secondo `resume` non può eseguirla due volte. Ma il secondo `resume` trova la pila vuota e la marca come finita (`is_done = true`). Quando la fiber in corso fa `yield`, la pila viene salvata, però `is_done` resta vero e la fiber non riparte più. Il commento nel codice dice che il secondo `resume` «non fa nulla».

Effetto. Un comando che fa `resume` della fiber di un worker mentre lo scheduler la sta eseguendo la uccide in silenzio.

Correzione proposta. Uno stato esplicito «in esecuzione», distinto da «finita». Un `resume` su una fiber in esecuzione restituisce `nil` o un errore senza toccarla.

INT-10 — Un `yield` dentro `condition-case` non sospende la fiber: diventa un errore e il gestore lo intercetta

Gravità **MEDIA**

Dove `lisp/eval.rs:1062` (`condition-case`), `lisp/utils.rs` (`condition_matches`)

Verifica Riprodotto: `(fiber (condition-case e (yield 1) (error 'h)) 3)` restituisce 3 al primo `resume` senza fermarsi; il gestore riceve `(error "YieldNotAllowed")`.

Cosa succede. Un `yield` è lecito solo dove il valutatore può registrare un punto di ripresa: un'istruzione del corpo o una forma di un `while`. Il corpo di `condition-case` non lo è, quindi il `yield` produce `YieldNotAllowed`. Il gestore generico `error` lo tratta come un errore qualsiasi.

Effetto. Una fiber che protegge con `condition-case` un passo che contiene un `yield` non si sospende mai. Il gestore d'errore parte e la fiber prosegue come se il passo fosse fallito. Il codice sembra corretto e si comporta in un altro modo.

Correzione proposta. Fare in modo che `condition_matches` non intercetti mai `YieldNotAllowed`, così l'errore arriva all'autore. Oppure aggiungere un tipo di frame per `condition-case`, così il `yield` diventa lecito anche lì.

INT-11 — Il backquote ignora la coda puntata, le mappe e l'annidamento

Gravità **MEDIA**

Dove `lisp/eval.rs:1195 (eval_backquote: le Cons e le mappe sono restituite senza modifiche)`

Verifica Riprodotto: ``(a . ,b)` dà `(a . (unquote b))` invece di `(a . 2)`; ``{k ,b}` dà `{k (unquote b)}`; un backquote annidato dà `UndefinedFunction("unquote")`.

Cosa succede. Il lettore produce una `Cons` per una lista puntata e una mappa per `{...}`, e `eval_backquote` elabora solo le forme proprie e i vettori. Un livello di annidamento è documentato come limite, ma la coda puntata e le mappe no.

Effetto. Le macro che costruiscono liste puntate o mappe con il backquote producono strutture che contengono ancora `unquote`, e falliscono più tardi, lontano dalla causa.

Correzione proposta. Elaborare la coda di una `Cons` e i valori di una mappa nello stesso modo degli elementi di una lista; per l'annidamento, contare i livelli come in Emacs.

INT-12 — `format` interpreta le sequenze con backslash della stringa di formato

Gravità **MEDIA**

Dove `lisp/base/strings.rs:264-330 (primitive_format)`, errore a `lisp/base/strings.rs:313`

Verifica Riprodotto: `(format "\\b%s\\b" "w")` e `(format "C:\\Users\\%s" "me")` falliscono con «Wrong escape character»; la sequenza `\n` scritta come due caratteri diventa un a capo.

Cosa succede. Il lettore ha già trasformato le sequenze di escape della stringa letterale. `format` le interpreta una seconda volta e rifiuta quelle che non conosce.

Effetto. Non si può costruire con `format` un'espressione regolare con `\b` o `\s`, né un percorso Windows. Il testo che arriva da una variabile viene alterato.

Correzione proposta. Togliere l'interpretazione delle sequenze da `format`, che è compito del lettore.

INT-13 — `make-string` non ha limiti: un conteggio enorme termina il processo

Gravità **MEDIA**

Dove `lisp/base/strings.rs:16 (primitive_make_string)`

Verifica Riprodotto: `(make-string (* 10000000 1000000) "a")` termina il processo (uscita 134); con `1e23` il fuzzer ottiene un panic «capacity overflow».

Cosa succede. La lunghezza richiesta va direttamente all'allocazione, senza controlli e senza carburante.

Effetto. Chiusura dell'editor. Lo stesso schema si ritrova nelle primitive dell'editor ([CMD-3](#)).

Correzione proposta. Un limite esplicito (per esempio la dimensione massima di un buffer) con un errore `args-out-of-range`, e `try_reserve` al posto dell'allocazione diretta.

INT-14 — I numeri interi oltre 2^{63} vengono stampati saturati

Gravità **BASSA**

Dove `lisp/base/mod.rs:236-238 (format_number: n as i64)`

Verifica Riprodotto: `(number-to-string 18446744073709551616)` dà `"9223372036854775807"`.

Cosa succede. Un numero intero viene stampato convertendolo a `i64`, conversione che satura.

Correzione proposta. Stampare come intero solo se `n.abs() < 2^53`, cioè quando la rappresentazione è esatta; altrimenti usare la forma decimale di `f64`.

INT-15 — `add-to-list` e funzioni affini: ambiente sbagliato, terzo argomento, costo quadratico

Gravità BASSA

Dove `lisp/base/lists.rs` (`add-to-list`, `append-to-list`, `remove-from-list`)

Verifica Riprodotto in una sessione precedente: chiamata dentro una funzione, `add-to-list` lascia invariata la lista globale. Gli altri punti vengono dalla lettura del codice.

Cosa succede. La lista viene riscritta con `set_variable` nell'ambiente del chiamante (INT-3). Il terzo argomento non ha il significato di `APPEND` che ha in Emacs: viene aggiunto come elemento. Con più elementi, questi sono aggiunti in ordine inverso, senza controllare i duplicati tra gli argomenti stessi. Il costo è $O(n \cdot m)$ e non consuma carburante.

Correzione proposta. Aggiornare la variabile dove è definita (`update_variable`); allineare la firma a quella di Emacs, (`add-to-list SYMBOL ELEMENT &optional APPEND`).

INT-16 — `memq` e `assq` confrontano con `equal`; `member` restituisce una copia

Gravità BASSA

Dove `lisp/base/lists.rs:421` e `:452` (`docstring`), `lisp/base/mod.rs:321-329` (`find_member`)

Verifica Dalla lettura del codice; (`memq "a" '("a")`) è vero mentre (`eq "a" "a"`) è falso.

Cosa succede. `memq` e `assq` usano lo stesso confronto strutturale di `member` e `assoc`, e la loro `docstring` dice ancora «eq is structural here», mentre `eq` ora confronta per identità. `member` e `memq` restituiscono una copia della coda (`proper_list(list[pos..].to_vec())`), non la coda condivisa: (`eq (member 2 l) (cdr l)`) è falso e il costo è $O(n)$.

Correzione proposta. Confronto con `eq` per `memq` e `assq`, `docstring` aggiornate, restituzione della coda condivisa scorrendo le `Cons`.

INT-17 — `string-to-number` accetta `inf` e `NaN`; `substring` oltre la fine restituisce la stringa vuota

Gravità BASSA

Dove `lisp/base/strings.rs`

Verifica Dalla lettura del codice.

Cosa succede. `string-to-number` usa `f64::from_str`, che accetta `"inf"`, `"NaN"` e `"1e5"`, mentre il lettore rifiuta tutte e tre le forme (LET-4). `substring` con un inizio oltre la fine della stringa restituisce `""` invece di un errore `args-out-of-range`.

Correzione proposta. Un unico analizzatore di numeri, condiviso dal lettore e da `string-to-number`; un errore per gli indici fuori dai limiti.

INT-18 — `spawn` restituisce una forma vuota che è `eq` a `nil` ma non `equal`

Gravità BASSA

Dove `lisp/eval.rs:475-510` (ramo `spawn`)

Verifica Riprodotto: (`list (eq (spawn ...) nil) (equal (spawn ...) nil) (null (spawn ...))`) dà (`t nil t`).

Cosa succede. Il valore restituito è `LispExp::form(vec![])`, cioè una lista vuota, invece del `nil` canonico. `is_nil` la considera `nil`, il confronto strutturale no.

Correzione proposta. Restituire `LispExp::nil()`.

INT-19 — Il controllo all'avvio lascia tre variabili globali

Gravità BASSA

Dove `lisp/handshake.rs:18-29`

Verifica Riprodotto: (`list verify-math fiber-step`) dà (`8 100`) in una sessione appena avviata.

Cosa succede. Il programma di verifica gira nell'ambiente globale e usa `setq`. In caso di fallimento, l'errore restituito è `UncorrectFunctionDefinition`, che non dice nulla del problema.

Correzione proposta. Eseguirlo in un ambiente figlio e restituire un errore che descriva il controllo fallito.

INT-20 — `UncorrectFunctionDefinition` viene usato per errori che non c'entrano

Gravità BASSA

Dove `lisp/base/fibers.rs:60-80`, `lisp/base/atoms.rs:54` e `:88` (lock avvelenato), `with-current-buffer` (argomento non funzione), `lisp/handshake.rs`

Verifica Riprodotto: `(with-current-buffer "**x*" (buffer-string))` dà `UncorrectFunctionDefinition`.

Cosa succede. Lo stesso errore copre un lock avvelenato, un argomento che non è una funzione e un controllo d'avvio fallito.

Correzione proposta. Usare `WrongArgumentType` per gli argomenti e un errore dedicato per i lock avvelenati.

INT-21 — `Atomi senza aggiornamento atomico`

Gravità BASSA

Dove `lisp/base/atoms.rs`

Verifica Dalla lettura del codice.

Cosa succede. Esistono solo `deref` e `reset`. Un incremento da due thread `((reset a (+ 1 (deref a))))` perde aggiornamenti, e nessuna primitiva permette di farlo in modo atomico.

Correzione proposta. Una primitiva `swap` che applica una funzione sotto il lock, oppure `compare-and-set`.

INT-22 — `Parametri chiamati t o nil` sono accettati ma irraggiungibili

Gravità BASSA

Dove `lisp/utils.rs:90` (`bind_lambda_args`), `lisp/eval.rs` (i simboli che valutano a sé stessi sono risolti prima della ricerca)

Verifica Dalla lettura del codice.

Cosa succede. `(lambda (t) t)` viene definita senza errori, ma nel corpo `t` vale sempre `t`, perché i simboli che valutano a sé stessi non vengono cercati nell'ambiente. Lo stesso vale per `nil` e per le keyword.

Correzione proposta. Rifiutare questi nomi in `parse_lambda_params`.

INT-23 — `function-doc` contiene un `unreachable!`

Gravità BASSA

Dove `lisp/base/functions.rs:186`

Verifica Dalla lettura del codice: oggi nessun percorso inserisce nello spazio delle funzioni qualcosa che non sia una lambda o una primitiva.

Cosa succede. Se un giorno lo spazio delle funzioni contenesse altro (per esempio un alias o una macro con un'altra rappresentazione), `function-doc` andrebbe in panic invece di restituire `nil`.

Correzione proposta. `_ => Ok(LispExp::nil())`.

4 Buffer, modifiche e annullamento

BUF-1 — `undo` e `redo` modificano il testo senza passare dalle due porte delle modifiche

Gravità **ALTA**

Dove `primitives/edits.rs:854-890` (`undo.undo(text)` e `undo.redo(text)` scrivono direttamente nel testo), `editor/replace.rs:217` (`replace_back`)

Verifica Riprodotto: in un file Rust, dopo `fn a() {}`, poi `//` all'inizio, poi `C-/`, la riga torna `fn a() {}` ma resta colorata interamente come commento, anche dopo 800 ms. In una sessione precedente: `undo` svuota un buffer in sola lettura.

Cosa succede. Ogni modifica dovrebbe passare da `insert_text` o `delete_range`. Sono loro che controllano la sola lettura, incrementano `version`, invalidano le cache di colorazione e di scansione e spostano marcatori, overlay e testo virtuale. `undo` e `redo` applicano invece le modifiche inverse direttamente sul testo con `UndoHistory::undo(text)` e poi si limitano a impostare `is_modified = true`.

Come riprodurlo.

```
in un file .rs vuoto: fn a() {} C-a // C-/  
→ il testo è di nuovo "fn a() {}", colorato come commento
```

Effetto. (1) `undo` modifica i buffer in sola lettura. (2) La versione non cambia, quindi la colorazione resta quella del testo precedente fino alla modifica successiva. La cache di scansione delle espressioni dà risposte sbagliate a `C-M-f`. Il salvataggio automatico crede di aver già salvato quella versione. (3) Overlay, testo virtuale e marcatori restano agli offset del testo vecchio, quindi le evidenziazioni cadono sulle parole sbagliate.

Correzione proposta. Far applicare a `UndoHistory` i suoi passi attraverso le due porte (o una terza porta che faccia le stesse cose senza registrare l'annullamento), e rifiutare `undo` in un buffer in sola lettura.

BUF-2 — `cursor_pos()` costa $O(\text{posizione})$: ogni fotogramma rilegge il buffer fino al punto

Gravità **MEDIA**

Dove `buffer/gap_buffer.rs:94-110`

Verifica Misurato su un buffer di 8 MB: `snapshot` impiega 7,9 ms con il punto alla fine e 0,03 ms con il punto all'inizio.

Cosa succede. Per trovare riga e colonna del punto `cursor_pos` scorre tutti i caratteri dall'inizio del buffer, anche se l'indice delle righe permetterebbe una ricerca binaria. `snapshot` la chiama per ogni finestra (posizione del cursore e riga di stato) e circa trenta altri punti la usano.

Effetto. In un file grande ogni tasto costa millisecondi in più, e il costo cresce con la distanza del punto dall'inizio.

Correzione proposta. Ricerca binaria della riga nell'indice dei ritorni a capo, poi un conteggio dei soli caratteri della riga.

BUF-3 — `GapBuffer::clone` è `todo!()` e `find_forward` va in `underflow` su un buffer vuoto

Gravità **BASSA**

Dove `buffer/gap_buffer.rs:691-695` (`Clone`), `buffer/gap_buffer.rs:182-196` (`find_forward`)

Verifica Dalla lettura del codice; oggi nessun percorso li raggiunge (`find_forward` è usata solo nei test).

Cosa succede. `BufferTrait` richiede `Clone`, ma l'implementazione per `GapBuffer` è un `todo!()`: il primo che clonerà un buffer manderà l'editor in panic. `find_forward` calcola `self.len() - 1`, che su un testo vuoto va in `underflow` (panic in debug) e poi arriva a un `unreachable!()`.

Correzione proposta. Derivare `Clone`, oppure togliere il vincolo dal trait; in `find_forward` usare `checked_sub`.

5 Tastiera, comandi e primitive dell'editor

CMD-1 — I caratteri non ASCII non si possono digitare né legare

Gravità **MEDIA**

Dove `input.rs:256` (for c in ' '..~': solo l'ASCII stampabile è legato a `self-insert`),
`primitives/mod.rs:70` (`s.len() == 1` conta byte, non caratteri)

Verifica Riprodotto: premendo è l'area messaggi mostra «è is undefined»; (`define-key nil "è" 'forward-char`) restituisce `nil`.

Cosa succede. La mappa globale lega a `self-insert` un tasto per ogni carattere da spazio a tilde; ogni altro carattere arriva alla risoluzione dei tasti senza un comando. `define-key` non può rimediare perché `parse_key` accetta un carattere singolo solo se è lungo un byte.

Effetto. L'editor è inutilizzabile per scrivere in italiano, francese, tedesco o in qualunque lingua con lettere accentate, e per i simboli tipografici (€, °, «»).

Correzione proposta. Un ripiego nella risoluzione dei tasti: un `KeyCode::Char` senza modificatori e senza un legame esplicito va a `self-insert`. In `parse_key`, `s.chars().count() == 1`.

CMD-2 — Mancano Canc, Inizio, Fine, PagSu, PagGiù e i tasti funzione

Gravità **MEDIA**

Dove `input.rs:5-17` (`KeyCode`), `src/tui.rs:60` (`translate_key` scarta tutto il resto)

Verifica Dalla lettura del codice: l'enum ha solo `Char`, `Backspace`, `Enter`, `Esc`, `Tab` e le quattro frecce.

Cosa succede. I tasti non rappresentati vengono scartati dalla TUI con una riga nel registro («no translation for the key»).

Effetto. Tasti che ogni utente si aspetta (Canc in particolare) non fanno nulla e non si possono legare.

Correzione proposta. Estendere `KeyCode` e `translate_key`, e aggiungere i nomi (`<delete>`, `<home>`, `<f1>` ...) a `parse_key`.

CMD-3 — Argomenti numerici enormi: allocazioni che terminano il processo e cicli senza fine

Gravità **MEDIA**

Dove `primitives/edits.rs:397-424` (`word_forward`, `word_backward`), `primitives/edits.rs:360-369` (`repeat_count`), `primitives/macros.rs:258-270` (`kmacro-insert-counter`), `indent-line-to`

Verifica Trovato dal fuzzer delle primitive (ogni funzione chiamata con 807 combinazioni di argomenti) e confermato a mano.

Cosa succede. Diverse primitive scritte in Rust usano un numero dell'utente come quantità da allocare o come numero di giri, senza un limite e senza carburante. `indent-line-to` crea una stringa di spazi lunga quanto la colonna richiesta. `kmacro-insert-counter` passa la larghezza a `format!`, che oltre 65 535 va in panic. `forward-word` e gli altri comandi sulle parole ripetono il passo COUNT volte senza fermarsi quando il punto ha raggiunto il bordo del buffer.

Come riprodurlo.

```
(indent-line-to 300000000000) ; abort: memory allocation failed
(kmacro-insert-counter 65536) ; panic: Formatting argument out of range
(forward-word 300000000) ; 0,6 s per 3e8 giri a vuoto
;; C-u 1000000000000 M-f → circa mezz'ora di blocco
```

Effetto. Un panic nel thread dei comandi chiude l'editor e lascia il terminale in raw mode (TUI-2); un ciclo lungo lo blocca senza che `C-g` possa interromperlo.

Correzione proposta. Uscire dai cicli quando il punto non si muove più; limitare larghezze e conteggi (per esempio a un milione) con un errore `args-out-of-range`; usare `try_reserve` per le allocazioni guidate dall'utente.

CMD-4 — Le combinazioni di modificatori in `define-key` sono limitate e un errore va solo nel registro

Gravità BASSA

Dove `primitives/mod.rs:28-80 (parse_key)`, `primitives/general.rs:43 (define-key)`

Verifica Riprodotto: "M-C-x" e "C-S-<up>" danno nil, "C-M-x" dà t.

Cosa succede. `parse_key` riconosce un solo prefisso tra C-M-, C-, M- e S-, quindi M-C-x (stesso tasto di C-M-x) e le combinazioni con Shift non si possono scrivere. Una sequenza non valida o un modo inesistente fanno restituire nil a `define-key`, con il motivo scritto solo nel registro.

Correzione proposta. Analizzare i prefissi in un ciclo, in qualsiasi ordine, e segnalare l'errore con un errore Lisp.

CMD-5 — `self-insert` ignora l'argomento prefisso

Gravità BASSA

Dove `input.rs:256-266` (il legame è `(self-insert "x")`, una forma che fornisce già i suoi argomenti), `primitives/edits.rs:200`

Verifica Riprodotto: C-u 3 x inserisce una x.

Cosa succede. Una forma che fornisce già i suoi argomenti non riceve l'argomento prefisso. Il manuale `man/editing.txt` descrive il comportamento di Emacs, cioè tre x.

Correzione proposta. Legare i caratteri a un comando che legge il tasto premuto (come `self-insert-command` di Emacs), oppure passare il prefisso alle forme di `self-insert`.

CMD-6 — Un argomento n non numerico diventa 0; un conteggio negativo diventa 0

Gravità BASSA

Dove `primitives/commands.rs:280 (parse::<f64>().unwrap_or(0.0))`, `primitives/edits.rs:360-369 (repeat_count)`

Verifica Dalla lettura del codice.

Cosa succede. Se l'utente scrive abc a una domanda numerica, il comando riceve 0 senza avviso. `repeat_count` trasforma un conteggio negativo in 0, quindi `(forward-word -2)` e M-- M-f non fanno nulla, mentre in Emacs vanno all'indietro.

Correzione proposta. Ripetere la domanda su un valore non numerico; interpretare i conteggi negativi come spostamenti nella direzione opposta.

6 Minibuffer

MIN-1 — Due domande aperte insieme condividono `*Minibuffer*` e lasciano una finestra fantasma

Gravità ALTA

Dove `editor/buffers.rs:259-300` (`close_buffer` chiude la **prima** finestra flottante che mostra il buffer),
`primitives/minibuffer.rs:322` (`default-minibuffer-prompt` usa sempre il nome `*Minibuffer*`)

Verifica Riprodotto con la sequenza `M-x one`, `M-x two`, `ESC`.

Cosa succede. Ogni prompt crea il buffer `*Minibuffer*` e una finestra flottante che lo mostra. Un secondo prompt aperto mentre il primo è ancora aperto (un `M-x` dentro `M-x`, o un comando che fa una domanda mentre un prompt è aperto) riusa lo stesso buffer, quindi il testo del primo si perde e le due finestre mostrano lo stesso testo. `ESC` chiude il buffer e la **prima** finestra trovata, cioè quella del prompt esterno. Resta così a schermo la finestra del prompt interno, senza un buffer valido: il fuoco torna a `*scratch*` e `ESC` risponde «`<esc>` is undefined». Il prompt successivo riusa il nome, e la finestra fantasma ne mostra il testo.

Effetto. Una finestra che non si può più chiudere copre parte dello schermo fino al riavvio, e la risposta che l'utente stava scrivendo nel prompt esterno è persa.

Correzione proposta. Nomi di buffer distinti per prompt (`*Minibuffer-1*` ...) e una pila di prompt, oppure rifiutare un secondo prompt finché il primo è aperto, come fa Emacs senza `enable-recursive-minibuffers`. In `close_buffer`, chiudere **tutte** le finestre flottanti che mostrano il buffer.

MIN-2 — `C-g` dentro il minibuffer non annulla la domanda

Gravità MEDIA

Dove `lisp/common-keymaps.lisp:290` (`C-g` → `keyboard-quit` anche nel minibuffer); manca un legame in `minibuffer-mode`

Verifica Riprodotto: `M-x` poi `C-g` scrive «Quit» e il prompt resta aperto; solo `ESC` lo chiude.

Cosa succede. `keyboard-quit` abbandona una sequenza di tasti a metà e azzerà l'argomento prefisso, ma non sa nulla dei prompt aperti.

Effetto. Chi viene da Emacs preme `C-g` per uscire da una domanda e non ottiene nulla; il prompt resta aperto e cattura i tasti successivi.

Correzione proposta. Legare `C-g` a `minibuffer-cancel` in `minibuffer-mode`.

MIN-3 — Il contenuto del minibuffer viene sostituito senza passare dalle porte delle modifiche

Gravità BASSA

Dove `feature/minibuffer.rs:46`

Verifica Dalla lettura del codice.

Cosa succede. La funzione che scrive la risposta proposta (completamento, cronologia) sostituisce il testo direttamente. Valgono le stesse conseguenze di [BUF-1](#): versione invariata e cache non invalidate. Nel minibuffer l'effetto visibile è modesto.

Correzione proposta. Usare `delete_range` e `insert_text`.

7 File e salvataggio

FIL-1 – Un salvataggio fallito con `C-x C-s` non viene segnalato

Gravità **ALTA**

Dove `primitives/io.rs:139-185 (save-buffer)`, `primitives/io.rs:1419-1441 (save-buffer--overwrite)`

Verifica Riprodotto: con un percorso in una directory inesistente l'area messaggi resta invariata e il buffer resta modificato. Anche il successo non viene detto: «Wrote ...» va solo nel registro.

Cosa succede. L'errore di `std::fs::write` viene passato a `log_diagnostic`, che scrive nel registro e non nell'area messaggi. `write-file`, al contrario, lo mostra.

Effetto. Chi salva su un disco pieno, su un file senza permessi di scrittura o su un percorso che non esiste più crede di aver salvato. L'unico avviso arriva all'uscita, con la domanda «... is unsaved. Quit anyway?», e solo se il buffer è ancora segnato come modificato.

Correzione proposta. Mostrare nell'area messaggi sia «Wrote PERCORSO» sia «Cannot save PERCORSO: motivo», come fa `write-file`.

FIL-2 – I file vengono scritti sul posto: un errore a metà li lascia troncati

Gravità **MEDIA**

Dove `primitives/io.rs:169, :370, :1427, :1513` e `modes/autosave.rs:239 (std::fs::write)`

Verifica Dalla lettura del codice.

Cosa succede. `std::fs::write` tronca il file e poi scrive. Se la scrittura si interrompe (disco pieno, processo terminato, alimentazione), sul disco resta un file troncato o vuoto, e la versione precedente è già persa.

Effetto. Possibile perdita del contenuto del file proprio quando qualcosa va storto.

Correzione proposta. Scrivere in un file temporaneo nella stessa directory, fare `sync_all` e poi `rename` sull'originale, conservando i permessi.

FIL-3 – Un file non UTF-8 non si apre e non viene detto nulla

Gravità **MEDIA**

Dove `editor/buffers.rs:21-45 (new_buffer: read_to_string)`, errore solo nel registro

Verifica Riprodotto con un file Latin-1: area messaggi e buffer corrente invariati, `find-file` restituisce `nil`.

Cosa succede. `std::fs::read_to_string` rifiuta qualsiasi byte non UTF-8; l'errore finisce nel registro e il comando non fa nulla di visibile.

Effetto. File in Latin-1, file binari e file con un solo byte sbagliato sembrano non aprirsi «per nessun motivo».

Correzione proposta. Un messaggio nell'area messaggi; in prospettiva un'apertura con `String::from_utf8_lossy` in un buffer in sola lettura, o una decodifica Latin-1 di ripiego.

FIL-4 – Una modifica fatta durante il salvataggio può restare non salvata ma non segnata

Gravità **BASSA**

Dove `primitives/io.rs:139-185 (save-buffer)`, `primitives/io.rs:1503 (write_buffer)`

Verifica Dalla lettura del codice.

Cosa succede. Il testo viene copiato sotto il lock, poi scritto senza lock, poi `is_modified` viene azzerato con un secondo lock. Una modifica arrivata nel frattempo (da un worker o da un callback) resta nel buffer ma il buffer risulta salvato.

Correzione proposta. Ricordare la `version` copiata e azzerare `is_modified` solo se è ancora quella.

FIL-5 – Il salvataggio automatico copia tutto il testo prima di sapere se serve, e segna come salvato anche un tentativo fallito

Gravità **BASSA**

Dove `modes/autosave.rs:200-251 (auto_save_for copia il testo con to_string() prima dei controlli; auto_saved_at impostato anche dopo un errore)`

Verifica Dalla lettura del codice; il secondo punto è documentato come «tentato».

Cosa succede. Ogni 30 secondi, per fino a quattro buffer, l'intero testo viene copiato sotto il lock di lettura anche quando il buffer non è modificato o è già stato salvato. Se la scrittura fallisce per un motivo passeggero, la versione viene comunque registrata e non si riprova finché il buffer non cambia.

Correzione proposta. Controllare `is_modified` e `auto_saved_at` prima di copiare; registrare la versione solo dopo una scrittura riuscita.

8 Avvio, configurazione e registro

AVV-1 – L'editor non parte se la directory corrente non è scrivibile

Gravità ALTA

Dove src/main.rs:21 (state.enable_log_file("rsedit.log")), editor/log.rs:14-18 (File::create)

Verifica Riprodotto lanciando l'editor da /proc: «Error: Os { code: 2, kind: NotFound ... }», uscita 1.

Cosa succede. Il registro viene creato con un percorso relativo, quindi nella directory da cui l'editor è lanciato, e un errore di creazione viene propagato con ? fino a main. File::create inoltre tronca il registro della sessione precedente.

Effetto. Chi lancia rsedit /etc/hosts da / o da una directory di sola lettura non riesce ad aprire nulla. In ogni progetto compare un file rsedit.log.

Correzione proposta. Mettere il registro nella directory di stato dell'utente (\$XDG_STATE_HOME/rsedit/ o accanto alla configurazione); se non si riesce ad aprirlo, continuare senza file di registro.

AVV-2 – Gli errori di sintassi nei file Lisp caricati vengono ignorati in silenzio

Gravità MEDIA

Dove editor/boot.rs:165-240 (eval_file: contenuto avvolto in (progn ...), errore del lettore → Ok(nil))

Verifica Riprodotto: con una parentesi in più il resto del file viene ignorato, con una parentesi non chiusa viene ignorato tutto. Un init.lisp la cui ultima riga è un commento senza a capo finale viene ignorato per intero; aggiungendo l'a capo funziona.

Cosa succede. Il file viene letto come un'unica forma (progn CONTENUTO). Una parentesi in più chiude la progn prima del tempo e il resto non viene mai letto. Una parentesi mancante, o un commento sull'ultima riga senza a capo (la parentesi aggiunta finisce dentro il commento), lascia la progn aperta. In entrambi i casi il lettore fallisce e eval_file restituisce nil senza dire nulla.

Come riprodurlo.

```
init.lisp:
  (setq my-config-loaded 42)
  ;; fine della configurazione      ← senza a capo finale
→ dopo l'avvio my-config-loaded non è definita, e nessun messaggio
```

Effetto. L'utente vede l'editor partire senza la sua configurazione, o con metà della configurazione, e non c'è nulla che indichi perché.

Correzione proposta. Leggere le forme una alla volta e valutarle in sequenza, segnalando nell'area messaggi la riga del primo errore di sintassi. Come minimo, format!("(progn\n{}}\n)").

AVV-3 – Un errore di scrittura del registro manda l'editor in panic

Gravità MEDIA

Dove editor/mod.rs:276-283 (log_diagnostic)

Verifica Dalla lettura del codice.

Cosa succede. Ogni diagnostica viene scritta nel file di registro con .expect("Failed to write into log file").

Effetto. Con il disco pieno, o se il file diventa non scrivibile, la diagnostica successiva chiude l'editor; e il primo tentativo di salvare su un disco pieno produce proprio una diagnostica.

Correzione proposta. Ignorare l'errore di scrittura del registro, o disattivare il file al primo errore, mantenendo le righe in memoria.

AVV-4 – Il registro in memoria cresce senza limite

Gravità BASSA

Dove managers/log.rs

Verifica Dalla lettura del codice.

Cosa succede. Ogni diagnostica viene conservata per tutta la sessione, e all-logs le copia tutte.

Correzione proposta. Un anello di dimensione fissa (per esempio le ultime 10 000 righe).

AVV-5 — La configurazione predefinita viene scritta solo se manca

Gravità BASSA

Dove editor/boot.rs:325-340 e :480

Verifica Dalla lettura del codice.

Cosa succede. Il file `init.lisp` predefinito carica i moduli e imposta i legami. Una volta scritto non viene più toccato, quindi chi lo ha già non riceve i moduli aggiunti nelle versioni successive.

Correzione proposta. Separare la parte predefinita, caricata sempre dal codice o da un file di sistema, dalla parte personale, che è l'unica scritta nella directory dell'utente.

9 Lavoro in background

BKG-1 — Un panic in un lavoro di background uccide per sempre il thread condiviso

Gravità ALTA

Dove background/mod.rs:104-185 (il ciclo dello scheduler, retain_mut a riga 170, senza catch_unwind), primitives/workers.rs:82 e :246

Verifica Riprodotto: un worker contatore si ferma a 16 dopo
(background-call 'boom (lambda () (eval-string "[1 . 2]"))); (running-workers) elenca ancora ("boom" "counter"); define-worker e background-call restituiscono nil senza messaggio.

Cosa succede. Tutti i lavori a turni (colorazione, prescansione, controllo dei file, salvataggio automatico, worker Lisp e `background-call` con una funzione) girano sullo stesso thread. Un panic in uno qualsiasi di essi termina quel thread, e nessuno lo riavvia. Da quel momento l'invio di nuovo lavoro fallisce. I commenti dicono che questo «succede solo alla chiusura dell'editor».

Effetto. L'editor continua a funzionare ma senza colorazione, senza salvataggio automatico, senza controllo dei file cambiati su disco e senza worker; nessun messaggio lo dice. Il messaggio del panic viene stampato su stderr sopra lo schermo della TUI. Se il panic avviene con un lock in scrittura tenuto, il lock resta avvelenato e il thread principale va in panic al primo `.expect("write lock on ...")`.

Correzione proposta. Eseguire ogni turno dentro `std::panic::catch_unwind`, ritirare solo il lavoro che ha fallito e scriverlo nel registro; in più, eliminare le cause note ([LET-2](#), [TUI-5](#)).

10 Modi, sintassi e colorazione

SIN-1 — Una regione di sintassi che corrisponde alla stringa vuota blocca il thread di background

Gravità **MEDIA**

Dove primitives/modes.rs:167-205 (add-syntax-region non controlla i pattern), modes/syntax.rs:333-393 (highlight_line non avanza dopo un'apertura, una chiusura o un escape vuoti)

Verifica Riprodotto: con BEGIN "x*" ed END "y*" un thread resta al 97 % di CPU e la colorazione di un file di una riga non finisce mai. Con BEGIN vuoto e NESTABLE la memoria cresce di circa 90 MB al secondo. Con un pattern normale lo stesso file si colora in 20 ms.

Cosa succede. highlight_line avanza la posizione al punto in cui finisce l'evento trovato. Per una regola protegge il caso vuoto (`pos = resume.max(pos + 1)`), per regioni ed escape no. Una regione che si apre e si chiude con due corrispondenze vuote nella stessa posizione si riapre all'infinito; una regione annidabile con un BEGIN vuoto continua a impilare stati.

Come riprodurlo.

```
(add-syntax-region 'rust-mode "x*" "y*" 'comment)
;; poi aprire un file .rs qualsiasi
;; errore di battitura realistico: BEGIN "#*" con END "$"
```

Effetto. Il thread condiviso resta occupato per sempre (BKG-1 per le conseguenze) oppure il processo esaurisce la memoria. Le grammatiche fornite con l'editor non hanno il problema; basta però un errore di battitura in una grammatica scritta dall'utente.

Correzione proposta. Rifiutare in add-syntax-region i pattern per cui is_match("") è vero, e in highlight_line forzare l'avanzamento di almeno un carattere dopo un evento vuoto.

SIN-2 — Colorare una riga molto lunga costa un tempo quadratico nella sua lunghezza

Gravità **MEDIA**

Dove modes/syntax.rs:444-476 (top_level_event riprova ogni regione e ogni regola dalla posizione corrente)

Verifica Misurato su una riga Rust ripetuta (`let a = "x";`): 65 KB in 0,96 s, 130 KB in 3,7 s, 260 KB in 14,8 s; una riga di 1 MB (un file minificato) richiederebbe circa quattro minuti.

Cosa succede. A ogni evento la scansione chiede a ogni pattern la sua prossima corrispondenza a partire dalla posizione corrente. I pattern che non trovano nulla scorrono ogni volta fino alla fine della riga, quindi il lavoro è proporzionale a eventi × lunghezza. Una riga non si può dividere tra due turni, perché LINES_PER_TURN conta righe.

Effetto. Mentre il thread condiviso colora la riga, salvataggio automatico, controllo dei file e worker sono fermi.

Correzione proposta. Ricordare per ogni pattern la prossima corrispondenza trovata e ricalcolarla solo quando la posizione la supera; oltre una certa lunghezza, non colorare la riga.

SIN-3 — write-file che cambia il modo lascia i colori del modo precedente

Gravità **BASSA**

Dove primitives/io.rs:385-392

Verifica Dalla lettura del codice.

Cosa succede. Se il nuovo nome del file corrisponde a un altro modo, current_mode cambia ma la cache di colorazione non viene invalidata. I colori della grammatica precedente restano finché il testo non viene modificato. Non esiste inoltre una primitiva per cambiare il modo di un buffer già aperto.

Correzione proposta. Invalidare SyntaxCache e ScanCache a ogni cambio di modo; aggiungere una primitiva set-major-mode.

SIN-4 — *make-mode* su un modo esistente lo sostituisce con uno vuoto

Gravità **BASSA**

Dove `primitives/modes.rs:13-30 (modes.insert(mode_name, MajorMode::new(...))`

Verifica Dalla lettura del codice.

Cosa succede. Rivalutare il file di un modo (per esempio dopo una modifica) azzerla la sua mappa dei tasti, i suoi hook, la grammatica e la tabella di sintassi, compresi quelli che altri moduli avevano aggiunto (come l'hook di `electric-pair`).

Correzione proposta. Se il modo esiste già, restituirlo senza modificarlo, oppure svuotare solo grammatica e tabella.

SIN-5 — *Il registro delle facce cresce per sempre e l'indice è a 16 bit*

Gravità **BASSA**

Dove `ui/faces.rs:150-175`

Verifica Dalla lettura del codice.

Cosa succede. Ogni nome di faccia mai usato resta registrato. `(names.len() - 1) as u16` si avvolge dopo 65 536 nomi distinti, e un nome nuovo prenderebbe l'indice di uno vecchio.

Correzione proposta. Un controllo con errore oltre `u16::MAX`, o un indice a 32 bit.

SIN-6 — *forward-sexp dall'interno di un simbolo salta alla fine dell'espressione successiva*

Gravità **MEDIA**

Dove `modes/sexp.rs:563-590 (forward)`, `modes/sexp.rs:525-531 (run_to)`

Verifica Riprodotto: in `(foo bar) (baz)`, da 2 (dentro `foo`) `forward-sexp` porta a 8 (fine di `bar`) invece che a 4; da 13 (dentro `baz`) non si muove invece di arrivare a 14.

Cosa succede. `forward` porta la scansione fino a `from` con `run_to`, poi conta le espressioni completate da lì in avanti. Ma un passo della scansione consuma un intero simbolo: se `from` cade dentro un simbolo, `run_to` lo completa (e lo scarta) prima di restituire il controllo, e il primo completamento contato è quello dell'espressione dopo. Sul simbolo finale di una lista non resta nulla da contare a quel livello e il punto non si muove.

Effetto. `C-M-f` e `kill-sexp` (che usa la stessa funzione) dall'interno di una parola si comportano in modo diverso da Emacs: il primo salta un'espressione, il secondo cancella anche l'espressione successiva.

Correzione proposta. In `run_to` fermarsi prima di un passo che supererebbe il limite, oppure in `forward` controllare se l'ultimo completamento di `run_to` finisce dopo `from` e contarlo.

11 Ricerca, sostituzione e liste di risultati

RIC-1 — Una sostituzione con un'espressione che può essere vuota mette tutte le sostituzioni nello stesso punto

Gravità **MEDIA**

Dove `text/search.rs:902-919` (`Replace::accept`), `editor/replace.rs:184` (`replace_rest`)

Verifica Riprodotto: (`replace-regex "x*" "-"`) su `abc` dà `-----abc` invece di `-a-b-c-`.

Cosa succede. Dopo una sostituzione di una corrispondenza vuota, la ricerca riprende da `found.start + new_len`, cioè subito dopo il testo appena inserito e **prima** del carattere successivo. Lì trova un'altra corrispondenza vuota, e così via. Si ferma solo per il tetto di sicurezza di `len + 1` sostituzioni.

Correzione proposta. Per una corrispondenza vuota riprendere da `found.start + new_len + 1`, scavalcando un carattere, come fa Emacs.

RIC-2 — La ricerca all'indietro con un'espressione regolare costa un tempo quadratico

Gravità **MEDIA**

Dove `text/search.rs:157-188` (`search_backward`, ramo `Regex`), usata da `feature/isearch.rs:134`

Verifica Misurato con `C-M-r` dalla fine di un file `ab ab ab ...`, per il solo tasto `b`: 15 KB in 0,24 s, 30 KB in 0,99 s, 60 KB in 3,59 s.

Cosa succede. Il motore delle espressioni cerca solo in avanti, quindi `search_backward` chiama `search_forward` in un ciclo e tiene l'ultima corrispondenza trovata. Ma ogni chiamata a `search_forward` copia l'intero buffer in una `String` e converte gli offset con una scansione lineare. È lo stesso costo quadratico che il commento sopra `Pattern::scan` dice di evitare.

Effetto. Su un file di qualche centinaio di KB con molte corrispondenze, ogni tasto premuto durante `C-M-r` blocca l'editor per minuti.

Correzione proposta. Una sola copia del testo e un solo passaggio con `captures_iter` che tiene l'ultima corrispondenza che finisce prima del limite.

RIC-3 — Ogni sostituzione è un gruppo di annullamento a sé

Gravità **BASSA**

Dove `editor/replace.rs:138-160` (`replace_this` chiama `undo.boundary()` a ogni sostituzione)

Verifica Riprodotto: dopo «Replaced 4 occurrences» servono quattro `C-/` per tornare al testo di partenza.

Cosa succede. Il confine di annullamento serve a `query-replace` per tornare indietro di una sostituzione, ma viene messo anche nelle sostituzioni non interattive.

Correzione proposta. Nessun confine tra le sostituzioni di `replace-string` e `replace-regex`; un confine solo nel percorso interattivo.

RIC-4 — Limiti della ricerca confinata a una regione e delle scansioni su righe lunghe

Gravità **BASSA**

Dove `text/search.rs:118-150` (`search_forward` con `LIMIT`), `text/search.rs:310-312` (`Cursor::column`)

Verifica Dalla lettura del codice.

Cosa succede. Se la prima corrispondenza dopo il punto finisce oltre il limite, viene scartata invece di cercarne una più corta: `a+` su `aaaa` con la regione `[0, 2)` non trova nulla, anche se la regione contiene `aa`. Nella scansione completa (`occur`, `grep`), `Cursor::column` ricalcola i caratteri dall'inizio della riga per ogni corrispondenza, quindi una riga lunghissima con molte corrispondenze costa un tempo quadratico.

Correzione proposta. Cercare nella sola fetta della regione; tenere la colonna in modo incrementale come già si fa per la riga.

RIC-5 — I risultati di `occur`, `grep` e `compile` puntano a righe che si spostano

Gravità **BASSA**

Dove `text/results.rs` (`Entry`), `primitives/results.rs:455-470`

Verifica Dalla lettura del codice.

Cosa succede. Ogni risultato conserva riga e offset come numeri. Dopo una modifica che aggiunge o toglie righe sopra un risultato, `next-error` porta alla riga sbagliata; Emacs usa marcatori che seguono il testo.

Correzione proposta. Marcatori nei buffer aperti, aggiornati dalle due porte delle modifiche.

RIC-6 — Alla fine di una ricerca incrementale il mark non è dove la ricerca è cominciata

Gravità BASSA

Dove `feature/isearch.rs:113-120 (show)`, `primitives/isearch.rs (isearch_exit)`

Verifica Riprodotto: in `hello world foo bar` con il punto a 0, `C-s foo RET` lascia il punto a 15 e il mark a 12, e l'area dei messaggi dice «Mark saved where search started».

Cosa succede. Durante la ricerca `show` usa il mark per evidenziare la corrispondenza come regione: lo mette all'estremità vicina del match. `isearch-exit` disattiva il mark ma non lo riporta su `origin`. Il messaggio è quello di Emacs, dove però il mark viene davvero salvato al punto di partenza, e `C-x C-x` o `C-u C-SPC` ci riportano.

Effetto. Dopo una ricerca non si può tornare al punto di partenza con il mark, e una regione costruita dopo la ricerca parte dall'inizio del match invece che dal punto di partenza.

Correzione proposta. In `isearch_exit` mettere il mark (inattivo) su `session.origin` quando è diverso dal punto finale, come fa Emacs.

12 Comandi di shell

SHL-1 — Un comando che scrive molto su stderr si blocca

Gravità **MEDIA**

Dove `primitives/shell.rs:76-110` (`ShellTask::execute`)

Verifica Riprodotto in una sessione precedente con uno script che scrive più di 64 KB su stderr.

Cosa succede. Lo stdout viene letto fino alla fine, e solo dopo lo stderr. Se il processo riempie il buffer della pipe di stderr (circa 64 KB) prima di chiudere stdout, resta fermo a scrivere su stderr mentre l'editor aspetta stdout: nessuno dei due può proseguire.

Effetto. Un `M-x compile` con molti avvisi del compilatore (che vanno su stderr) non finisce mai; il buffer dei risultati resta a metà e il contatore dei comandi in corso tiene sveglio il disegno.

Correzione proposta. Leggere stderr su un secondo thread, o unire i due flussi (`2>&1`) già nella shell.

SHL-2 — Un solo byte non UTF-8 nell'output interrompe il comando

Gravità **MEDIA**

Dove `primitives/shell.rs:79-95`

Verifica Riprodotto: con `printf 'one\n\377\ntwo\n'; sleep 0.3; echo after` il buffer contiene «one», «[unreadable output: stream did not contain valid UTF-8]» e «— killed —»; «two», «after» e lo stderr sono persi.

Cosa succede. Alla prima riga non UTF-8 il ciclo di lettura esce. La pipe di stdout viene chiusa, e alla scrittura successiva il processo riceve SIGPIPE e muore.

Effetto. Una compilazione che stampa un nome di file in Latin-1, o un `grep` che incontra un file binario, viene uccisa a metà.

Correzione proposta. Leggere byte con `read_until(b'\n')` e convertire ogni riga con `String::from_utf8_lossy`.

SHL-3 — Non c'è modo di fermare un comando in corso

Gravità **BASSA**

Dove `primitives/shell.rs` (`ShellTask` possiede il `Child`)

Verifica Dalla lettura del codice.

Cosa succede. Il processo figlio appartiene al thread che ne legge l'output e nessuna primitiva lo raggiunge. Uccidere il buffer non uccide il processo: `append_to_buffer` restituisce `false`, il valore viene ignorato e il processo continua fino alla fine.

Correzione proposta. Un registro dei processi per nome di buffer e una primitiva `kill-process`; alla chiusura del buffer, inviare SIGTERM.

SHL-4 — Il nome di una pagina di manuale viene passato alla shell senza virgolette

Gravità **BASSA**

Dove `core/lisp/manpage.lisp:93-101` e `:69-79`

Verifica Dalla lettura del codice.

Cosa succede. `(format "man -w %s 2>/dev/null" name)` passa il nome così com'è a `/bin/sh -c`. Un nome con spazi o metacaratteri viene interpretato dalla shell: `printf(3)` dà un errore di sintassi, che si presenta come «No manual entry», e `a;b` esegue due comandi. `manpage-at-point` è limitato ai caratteri di un simbolo, ma `*` viene espanso dalla shell. Anche il percorso della pagina interna viene costruito concatenando il nome.

Correzione proposta. Racchiudere il nome tra apici singoli, con gli eventuali apici interni preceduti da escape, oppure rifiutare i nomi che non sono `[A-Za-z0-9._+-]+`.

13 Disegno e terminale

TUI-1 — I caratteri di controllo del testo arrivano al terminale: un file può comandarlo

Gravità ALTA

Dove `ui/layout.rs:219-224` (`visible_text` copia i caratteri senza filtrarli), `src/tui.rs:586-618` (`draw_clipped_row` li stampa con `Print`)

Verifica Riprodotto: in un file che contiene `ESC J 52 ; c ; aGk= ESC \` la riga della fotografia contiene la sequenza intatta (`"\u{1b}J52;c;aGk=\u{1b}\\"`), che la TUI scrive sul terminale.

Cosa succede. Tra il buffer e il terminale nessuno sostituisce i caratteri di controllo (ESC, BEL, CR, BS, DEL, i controlli C1). Il terminale li esegue.

Effetto. Aprire un file, guardare l'output di un comando, un risultato di grep o una pagina di manuale che contengono sequenze di escape fa eseguire quelle sequenze al terminale. OSC 52 riscrive gli appunti di sistema: l'utente incollerà poi in una shell un comando scelto da chi ha preparato il file. OSC 0 cambia il titolo della finestra; le sequenze CSI spostano il cursore, cancellano lo schermo o cambiano i colori. Anche senza malizia, un `\r` o un `\b` in un file rovinano la visualizzazione.

Correzione proposta. Al momento del disegno, sostituire i controlli (U+0000–U+001F tranne il tab, U+007F, U+0080–U+009F) con una rappresentazione visibile, come `^[` in Emacs o il carattere `ESC`, contandone la larghezza nella mappa delle colonne.

TUI-2 — Un panic o un errore di I/O lasciano il terminale in raw mode

Gravità ALTA

Dove `src/tui.rs:681-835` (`tui_main`), `src/main.rs` (nessun `panic::set_hook`, nessun `catch_unwind`)

Verifica Dalla lettura del codice (nessuna occorrenza di `set_hook` o `catch_unwind` nella TUI); i panic che lo innescano sono verificati altrove ([LET-2](#), [CMD-3](#)).

Cosa succede. `tui_main` attiva il raw mode, il bracketed paste e, a richiesta, la cattura del mouse, e li disattiva solo sul percorso di uscita normale. Un panic risale fuori da `main` senza ripristinarli; ogni `?` su un errore di I/O nel ciclo (`render_frame`, `read`, `poll`) esce da `tui_main` saltando il ripristino.

Effetto. Dopo un crash la shell non mostra ciò che si digita, Invio non funziona e i clic del mouse stampano sequenze di escape. L'utente deve digitare `reset` alla cieca. Il messaggio del panic stesso viene stampato «a scalini», in raw mode.

Correzione proposta. Una struttura di guardia il cui `Drop` ripristina il terminale, più un `panic::set_hook` che ripristina prima di stampare il messaggio.

TUI-3 — Le tabulazioni non vengono espanse

Gravità MEDIA

Dove `ui/layout.rs` (nessuna gestione del tab), `src/tui.rs`

Verifica Riprodotto: la riga della fotografia contiene il carattere `\t`.

Cosa succede. L'editor conta un tab come una colonna. Il terminale invece lo espande fino alla tabulazione successiva, quindi il resto della riga viene disegnato più a destra di dove l'editor crede. Il commento di `text/rectangle.rs:15` («a tab is drawn in one cell») descrive il modello interno, non ciò che accade sullo schermo.

Effetto. In un Makefile, in un file Go o in qualsiasi sorgente indentato con tab, il cursore appare nel punto sbagliato, i colori cadono sulle parole sbagliate e il resto della riga precedente può restare visibile.

Correzione proposta. Espandere i tab in spazi nella composizione delle righe (`compose_row`), con una larghezza configurabile, e usare la stessa mappa per il cursore e per i clic del mouse.

TUI-4 — I caratteri larghi contano come una colonna

Gravità MEDIA

Dove `ui/layout.rs`, `src/tui.rs`

Verifica Dalla lettura del codice: non si usa la larghezza di visualizzazione (`unicode-width`).

Cosa succede. Un ideogramma o un'emoji occupa due celle del terminale, ma l'editor ne conta una.

Effetto. Come per i tab: cursore, evidenziazioni, ritaglio e riempimento delle righe sfasati nei testi in cinese, giapponese, coreano o con emoji.

Correzione proposta. Calcolare la larghezza con `unicode-width` nella stessa mappa di [TUI-3](#).

TUI-5 — Un colore esadecimale con caratteri non ASCII manda l'editor in panic

Gravità MEDIA
Dove `ui/faces.rs:241-255`
Verifica Riprodotto: `(set-face 'region "#aéb12")` termina il programma con «end byte index 2 is not a char boundary» (uscita 101).

Cosa succede. `Color::parse` controlla la lunghezza in byte e poi taglia la stringa con `&hex[0..2]`. Con un carattere di due byte il taglio cade a metà del carattere.

Effetto. Chiusura dell'editor. Il caso si presenta anche con un file di tema sbagliato, e se il colore viene applicato da un lavoro di background uccide anche lo scheduler ([BKG-1](#)).

Correzione proposta. Rifiutare i colori che non sono ASCII (`hex.is_ascii()`) prima di tagliare.

TUI-6 — Le finestre flottanti non mostrano colori di sintassi né overlay

Gravità BASSA
Dove `editor/frame.rs:136-171`
Verifica Dalla lettura del codice: per una finestra flottante si raccolgono solo la regione e il testo virtuale.

Cosa succede. Un buffer mostrato in una finestra flottante appare senza colori e senza le evidenziazioni degli overlay; lo stesso buffer in una finestra affiancata le ha.

Correzione proposta. Usare per le finestre flottanti la stessa raccolta delle evidenziazioni delle finestre affiancate.

TUI-7 — %p nella riga di stato dice All solo per un buffer di una riga

Gravità BASSA
Dove `ui/windows.rs:1359-1372 (position_in_buffer)`
Verifica Dalla lettura del codice.

Cosa succede. Il commento dice «All quando l'intero buffer sta in una schermata», ma il codice verifica `lines <= 1`.

Correzione proposta. Passare l'altezza della finestra e confrontare con quella.

TUI-8 — La TUI non usa lo schermo alternativo

Gravità BASSA
Dove `src/tui.rs:681-835`
Verifica Dalla lettura del codice: nessun `EnterAlternateScreen`.

Cosa succede. L'editor disegna sullo schermo principale del terminale e all'uscita lo cancella con `Clear(All)`.

Effetto. Il contenuto che il terminale mostrava prima di avviare l'editor, per esempio l'output di un comando, va perso.

Correzione proposta. `EnterAlternateScreen` all'avvio e `LeaveAlternateScreen` all'uscita, nella stessa guardia di [TUI-2](#).

14 Documentazione, commenti, codice morto, build e test

DOC-1 — Commenti e docstring che descrivono un comportamento che non c'è più

Gravità BASSA
Dove vedi l'elenco
Verifica Dalla lettura del codice.

Cosa succede.

- `feature/minibuffer.rs:15` e `src/tui.rs:401`: il minibuffer «docked to the last few lines of the frame».
- `editor/buffers.rs:269`: «this editor does not yet ask before that kill», mentre `kill-buffer` ora chiede conferma.
- `primitives/workers.rs:82` e `:246`: «The scheduler is gone, which happens only as the editor shuts down», falso dopo un panic ([BKG-1](#)).
- `buffer/undo.rs:216`: «How many groups this history keeps», mentre il limite è in byte di testo cancellato.
- `primitives/shell.rs:215` (`shell-command-to-string`): «the background worker cannot call back into Lisp», superato da `background-call` con `ON-DONE`.
- `text/rectangle.rs:15`: «a tab is drawn in one cell» ([TUI-3](#)).
- `lisp/base/atoms.rs:37` e `:69`: gli esempi usano `(atom 0)`, ma il costruttore è `make-atom`.
- `lisp/base/lists.rs:423` e `:454`: «eq is structural» ([INT-16](#)).
- `lisp/base/fibers.rs:55`: il secondo `resume` «does nothing» ([INT-9](#)).

Correzione proposta. Aggiornarli insieme alle correzioni dei problemi a cui rimandano.

DOC-2 — Codice morto e trappole latenti

Gravità BASSA
Dove `buffer/mod.rs:27` (`local_keymap`), `editor/mod.rs:314-318` (`impl Debug for EditorState`)
Verifica Dalla lettura del codice.

Cosa succede. `Buffer::local_keymap` viene inizializzato ma mai letto. `Debug` per `EditorState` è un `todo!()`: un `dbg!`, un `{:?}` o un `assert_eq!` fallito su una struttura che contiene uno stato dell'editor va in panic invece di stampare.

Correzione proposta. Togliere il campo o implementarlo; un `Debug` che stampi almeno i nomi dei buffer.

DOC-3 — Lo script di build presume dove si trova la directory `target`

Gravità BASSA
Dove `core/build.rs:30-70`
Verifica Riprodotto: la compilazione in release di un crate che dipende da `rsedit_core` fallisce finché `target/release` non esiste.

Cosa succede. I moduli Lisp vengono copiati in `$CARGO_WORKSPACE_DIR/target/$PROFILE/data`, ignorando `CARGO_TARGET_DIR`, `--target-dir` e `target/<triple>`. `fs::create_dir` fallisce se la directory genitore non esiste ancora, e il fallimento è un panic. `CARGO_WORKSPACE_DIR` viene da `.cargo/config.toml` ed è letto con `unwrap`.

Effetto. Con una directory di build diversa l'editor si avvia senza moduli; da un altro crate la build fallisce.

Correzione proposta. Usare `OUT_DIR` oppure incorporare i moduli con `include_str!`; in alternativa `create_dir_all`.

DOC-4 — Un test di prestazioni instabile e clippy che fallisce sui test

Gravità BASSA
Dove `core/src/tests/perf/mod.rs:256`, `core/src/lisp/tests/parser_tests.rs:163`
Verifica Riprodotto: su 1 986 test ne fallisce uno, `perf::suite` (confronto di tempi 0,99×).
`cargo clippy --all-targets` si ferma su `approx_constant (-3.14)`, una lint di livello deny.

Cosa succede. Il test confronta tempi misurati, che su una macchina condivisa variano. Il letterale `-3.14` in un test del lettore viene scambiato da clippy per π .

Correzione proposta. Escludere i test di prestazioni dalla suite normale (`#[ignore]` o una feature); usare un altro letterale, per esempio `-2.5`.

15 Appendici

15.1 Differenze da Emacs che non sono difetti

Alcuni comportamenti differiscono da Emacs per scelta, o comunque senza produrre risultati sbagliati. Sono elencati qui perché chi conosce Emacs non li scambi per errori.

- `%` ha la stessa semantica di `mod`: il segno del risultato segue il divisore. In Emacs `%` è il resto e segue il dividendo: `(% -7 2)` dà 1 qui e `-1` in Emacs.
- Le espressioni regolari usano la sintassi del crate `regex` di Rust, non quella di Emacs. `\(` è una parentesi letterale e i gruppi si scrivono `( )`. Non esistono lookaround né riferimenti all'indietro, ed è proprio ciò che garantisce un tempo lineare.
- `out-of-fuel` non si intercetta con `condition-case`. È voluto: un ciclo infinito non deve potersi proteggere dal meccanismo che lo ferma. I gestori di `unwind-protect` girano comunque.
- Le liste di risultati, le ricerche e le sostituzioni non tornano mai all'inizio da sole: quando la lista finisce lo dicono, come scelta esplicita di progetto.

15.2 Parti esaminate e trovate solide

Queste parti sono state esercitate di proposito sui casi limite senza trovare problemi:

- lo scanner delle espressioni bilanciate (`modes/sexp.rs`): stringhe con escape alla fine del buffer, parentesi sbilanciate in entrambe le direzioni, letterali carattere, lifetime di Rust, stringhe raw, commenti a blocco annidati, commenti che contengono parentesi;
- la ricerca incrementale: parentesi letterali, espressioni non valide, cancellazione, ricerca fallita e ripresa dopo il fallimento;
- `kill` e `yank`, compreso `M-y`, e le macro di tastiera (con il limite alla ricorsione delle macro);
- `kill-buffer` di un buffer mostrato in due finestre, la chiusura dell'unica finestra, la chiusura di `*scratch*`;
- la sostituzione `a` → `aa`, che termina;
- i comandi di modifica ai bordi del buffer;
- `delete-file` sui link simbolici a directory, che cancella il link e non il contenuto della directory di destinazione;
- l'uscita con buffer non salvati: dopo un salvataggio fallito il buffer resta segnato come modificato e l'editor chiede conferma prima di uscire;
- tutte le forme speciali dell'interprete con sintassi malformata (181 322 forme valutate dal fuzzer, nessun panic).